第19章 枚举类型

关键字enum可以将一组具名的值的有限集合创建为一种新的类型，而这些具名的值可以作为常规的程序组件使用。这是一种非常有用的功能[⊖]。

在第5章结束的时候，我们已经简单地介绍了枚举的概念。现在，你对Java已经有了更深刻的理解，因此可以更深入地学习Java SE5中的枚举了。你将在本章中看到，使用**enum**可以做很多有趣的事情，同时，我们也会深入其他的Java特性，例如泛型和反射。在这个过程中，我们还将学习一些设计模式。

19.1 基本enum特性

我们已经在第5章看到，调用**enum**的**values()**方法，可以遍历**enum**实例。**values()**方法返回**enum**实例的数组，而且该数组中的元素严格保持其在**enum**中声明时的顺序，因此你可以在循环中使用**values()**返回的数组。

创建**enum**时，编译器会为你生成一个相关的类，这个类继承自**java.lang.Enum**。下面的例子演示了**Enum**提供的一些功能：

```
//: enumerated/EnumClass.java
// Capabilities of the Enum class
import static net.mindview.util.Print.*;

enum Shrubbery { GROUND, CRAWLING, HANGING }

public class EnumClass {
  public static void main(String[] args) {
    for(Shrubbery s : Shrubbery.values()) {
      print(s + " ordinal: " + s.ordinal());
      printnb(s.compareTo(Shrubbery.CRAWLING) + " ");
      printnb(s.equals(Shrubbery.CRAWLING) + " ");
      print(s == Shrubbery.CRAWLING);
      print(s.getDeclaringClass());
      print(s.name());
      print("----------------------");
    }
    // Produce an enum value from a string name:
    for(String s : "HANGING CRAWLING GROUND".split(" ")) {
      Shrubbery shrub = Enum.valueOf(Shrubbery.class, s);
      print(shrub);
    }
  }
} /* Output:
GROUND ordinal: 0
-1 false false
class Shrubbery
GROUND
----------------------
CRAWLING ordinal: 1
0 true true
class Shrubbery
CRAWLING
```


⊖ Joshua Bloch为撰写此章提供了很大帮助。

```
---------------------
HANGING ordinal: 2
1 false false
class Shrubbery
HANGING
---------------------
HANGING
CRAWLING
GROUND
*///:~
```

ordinal()方法返回一个**int**值，这是每个**enum**实例在声明时的次序，从0开始。可以使用**==**来比较**enum**实例，编译器会自动为你提供**equals()**和**hashCode()**方法。**Enum**类实现了**Comparable**接口，所以它具有**compareTo()**方法。同时，它还实现了**Serializable**接口。

如果在**enum**实例上调用**getDeclaringClass()**方法，我们就能知道其所属的**enum**类。 1012

name()方法返回**enum**实例声明时的名字，这与使用**toString()**方法效果相同。**valueOf()**是在**Enum**中定义的**static**方法，它根据给定的名字返回相应的**enum**实例，如果不存在给定名字的实例，将会抛出异常。

19.1.1 将静态导入用于enum

先看一看第5章中**Burrito.java**的另一个版本：

```
//: enumerated/Spiciness.java
package enumerated;

public enum Spiciness {
  NOT, MILD, MEDIUM, HOT, FLAMING
} ///:~
```

```
//: enumerated/Burrito.java
package enumerated;
import static enumerated.Spiciness.*;

public class Burrito {
  Spiciness degree;
  public Burrito(Spiciness degree) { this.degree = degree;}
  public String toString() { return "Burrito is "+ degree;}
  public static void main(String[] args) {
    System.out.println(new Burrito(NOT));
    System.out.println(new Burrito(MEDIUM));
    System.out.println(new Burrito(HOT));
  }
} /* Output:
Burrito is NOT
Burrito is MEDIUM
Burrito is HOT
*///:~
```

使用**static import**能够将**enum**实例的标识符带入当前的命名空间，所以无需再用**enum**类型来修饰**enum**实例。这是一个好的想法吗？或者还是显式地修饰**enum**实例更好？这要看代码的复杂程度了。编译器可以确保你使用的是正确的类型，所以唯一需要担心的是，使用静态导入会不会导致你的代码令人难以理解。多数情况下，使用**static import**还是有好处的，不过，程序员还是应该对具体情况进行具体分析。 1013

注意，在定义**enum**的同一个文件中，这种技巧无法使用；如果是在默认包中定义**enum**，这种技巧也无法使用（在Sun内部对这一点显然也有不同意见）。

19.2 向enum中添加新方法

除了不能继承自一个**enum**之外，我们基本上可以将**enum**看作一个常规的类。也就是说，

我们可以向**enum**中添加方法。**enum**甚至可以有**main()**方法。

一般来说，我们希望每个枚举实例能够返回对自身的描述，而不仅仅只是默认的**toString()**实现，这只能返回枚举实例的名字。为此，你可以提供一个构造器，专门负责处理这个额外的信息，然后添加一个方法，返回这个描述信息。看一看下面的示例：

```
//: enumerated/OzWitch.java
// The witches in the land of Oz.
import static net.mindview.util.Print.*;

public enum OzWitch {
  // Instances must be defined first, before methods:
  WEST("Miss Gulch, aka the Wicked Witch of the West"),
  NORTH("Glinda, the Good Witch of the North"),
  EAST("Wicked Witch of the East, wearer of the Ruby " +
    "Slippers, crushed by Dorothy's house"),
  SOUTH("Good by inference, but missing");
  private String description;
  // Constructor must be package or private access:
  private OzWitch(String description) {
    this.description = description;
  }
  public String getDescription() { return description; }
  public static void main(String[] args) {
    for(OzWitch witch : OzWitch.values())
      print(witch + ": " + witch.getDescription());
  }
} /* Output:
WEST: Miss Gulch, aka the Wicked Witch of the West
NORTH: Glinda, the Good Witch of the North
EAST: Wicked Witch of the East, wearer of the Ruby
Slippers, crushed by Dorothy's house
SOUTH: Good by inference, but missing
*///:~
```

1014

注意，如果你打算定义自己的方法，那么必须在**enum**实例序列的最后添加一个分号。同时，Java要求你必须先定义**enum**实例。如果在定义**enum**实例之前定义了任何方法或属性，那么在编译时就会得到错误信息。

enum中的构造器与方法和普通的类没有区别，因为除了有少许限制之外，**enum**就是一个普通的类。所以，我们可以使用**enum**做许多事情（虽然，我们一般只使用普通的枚举类型）。

在这个例子中，虽然我们有意识地将**enum**的构造器声明为**private**，但对于它的可访问性而言，其实并没有什么变化，因为（即使不声明为**private**）我们只能在**enum**定义的内部使用其构造器创建**enum**实例。一旦**enum**的定义结束，编译器就不允许我们再使用其构造器来创建任何实例了。

19.2.1 覆盖enum的方法

覆盖**toSring()**方法，给我们提供了另一种方式来为枚举实例生成不同的字符串描述信息。在下面的示例中，我们使用的就是实例的名字，不过我们希望改变其格式。覆盖**enum**的**toSring()**方法与覆盖一般类的方法没有区别：

```
//: enumerated/SpaceShip.java
public enum SpaceShip {
  SCOUT, CARGO, TRANSPORT, CRUISER, BATTLESHIP, MOTHERSHIP;
  public String toString() {
    String id = name();
    String lower = id.substring(1).toLowerCase();
    return id.charAt(0) + lower;
  }
  public static void main(String[] args) {
```

```
    for(SpaceShip s : values()) {
      System.out.println(s);
    }
  }
} /* Output:
Scout
Cargo
Transport
Cruiser
Battleship
Mothership
*///:~
```

1015

toString()方法通过调用**name()**方法取得**SpaceShip**的名字，然后将其修改为只有首字母大写的格式。

19.3 switch语句中的enum

在**switch**中使用**enum**，是**enum**提供的一项非常便利的功能。一般来说，在**switch**中只能使用整数值，而枚举实例天生就具备整数值的次序，并且可以通过**ordinal()**方法取得其次序（显然编译器帮我们做了类似的工作），因此我们可以在**switch**语句中使用**enum**。

虽然一般情况下我们必须使用**enum**类型来修饰一个**enum**实例，但是在**case**语句中却不必如此。下面的例子使用**enum**构造了一个小型状态机：

```
//: enumerated/TrafficLight.java
// Enums in switch statements.
import static net.mindview.util.Print.*;

// Define an enum type:
enum Signal { GREEN, YELLOW, RED, }

public class TrafficLight {
  Signal color = Signal.RED;
  public void change() {
    switch(color) {
      // Note that you don't have to say Signal.RED
      // in the case statement:
      case RED:    color = Signal.GREEN;
                   break;
      case GREEN:  color = Signal.YELLOW;
                   break;
      case YELLOW: color = Signal.RED;
                   break;
    }
  }
  public String toString() {
    return "The traffic light is " + color;
  }
  public static void main(String[] args) {
    TrafficLight t = new TrafficLight();
    for(int i = 0; i < 7; i++) {
      print(t);
      t.change();
    }
  }
} /* Output:
The traffic light is RED
The traffic light is GREEN
The traffic light is YELLOW
The traffic light is RED
The traffic light is GREEN
The traffic light is YELLOW
The traffic light is RED
*///:~
```

1016

编译器并没有抱怨switch中没有default语句，但这并不是因为每一个Signal都有对应的case语句。如果你注释掉其中的某个case语句，编译器同样不会抱怨什么。这意味着，你必须确保自己覆盖了所有的分支。但是，如果在case语句中调用return，那么编译器就会抱怨缺少default语句了。这与是否覆盖了enum的所有实例无关。

练习1：(2) 修改TrafficLight.java，使用static import，使之无需用enum类型修饰其实例。

19.4 values()的神秘之处

前面已经提到，编译器为你创建的enum类都继承自Enum类。然而，如果你研究一下Enum类就会发现，它并没有values()方法。可我们明明已经用过该方法了，难道存在某种“隐藏的”方法吗？我们可以利用反射机制编写一个简单的程序，来查看其中的究竟：

```
//: enumerated/Reflection.java
// Analyzing enums using reflection.
import java.lang.reflect.*;
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

enum Explore { HERE, THERE }

public class Reflection {
  public static Set<String> analyze(Class<?> enumClass) {
    print("----- Analyzing " + enumClass + " -----");
    print("Interfaces:");
    for(Type t : enumClass.getGenericInterfaces())
      print(t);
    print("Base: " + enumClass.getSuperclass());
    print("Methods: ");
    Set<String> methods = new TreeSet<String>();
    for(Method m : enumClass.getMethods()
      methods.add(m.getName());
    print(methods);
    return methods;
  }
  public static void main(String[] args) {
    Set<String> exploreMethods = analyze(Explore.class);
    Set<String> enumMethods = analyze(Enum.class);
    print("Explore.containsAll(Enum)? " +
      exploreMethods.containsAll(enumMethods));
    printnb("Explore.removeAll(Enum): ");
    exploreMethods.removeAll(enumMethods);
    print(exploreMethods);
    // Decompile the code for the enum:
    OSExecute.command("javap Explore");
  }
} /* Output:
----- Analyzing class Explore -----
Interfaces:
Base: class java.lang.Enum
Methods:
[compareTo, equals, getClass, getDeclaringClass, hashCode,
name, notify, notifyAll, ordinal, toString, valueOf,
values, wait]
----- Analyzing class java.lang.Enum -----
Interfaces:
java.lang.Comparable<E>
interface java.io.Serializable
Base: class java.lang.Object
Methods:
[compareTo, equals, getClass, getDeclaringClass, hashCode,
name, notify, notifyAll, ordinal, toString, valueOf, wait]
```

1017

```
Explore.containsAll(Enum)? true
Explore.removeAll(Enum): [values]
Compiled from "Reflection.java"
final class Explore extends java.lang.Enum{
    public static final Explore HERE;
    public static final Explore THERE;
    public static final Explore[] values();
    public static Explore valueOf(java.lang.String);
    static {};
}
*///:~
```

1018

答案是，**values()**是由编译器添加的**static**方法。可以看出，在创建**Explore**的过程中，编译器还为其添加了**valueOf()**方法。这可能有点令人迷惑，**Enum**类不是已经有**valueOf()**方法了吗。不过**Enum**中的**valueOf()**方法需要两个参数，而这个新增的方法只需一个参数。由于这里使用的**Set**只存储方法的名字，而不考虑方法的签名，所以在调用**Explore.removeAll(Enum)**之后，就只剩下**[values]**了。

从最后的输出中可以看到，编译器将**Explore**标记为**final**类，所以无法继承自**enum**。其中还有一个**static**的初始化子句，稍后我们将学习如何重定义该句。

由于擦除效应（在第15章中介绍过），反编译无法得到**Enum**的完整信息，所以它展示的**Explore**的父类只是一个原始的**Enum**，而非事实上的**Enum<Explore>**。

由于**values()**方法是由编译器插入到**enum**定义中的**static**方法，所以，如果你将**enum**实例向上转型为**Enum**，那么**values()**方法就不可访问了。不过，在**Class**中有一个**getEnumConstants()**方法，所以即便**Enum**接口中没有**values()**方法，我们仍然可以通过**Class**对象取得所有**enum**实例：

```
//: enumerated/UpcastEnum.java
// No values() method if you upcast an enum

enum Search { HITHER, YON }

public class UpcastEnum {
  public static void main(String[] args) {
    Search[] vals = Search.values();
    Enum e = Search.HITHER; // Upcast
    // e.values(); // No values() in Enum
    for(Enum en : e.getClass().getEnumConstants())
      System.out.println(en);
  }
} /* Output:
HITHER
YON
*///:~
```

1019

因为**getEnumConstants()**是**Class**上的方法，所以你甚至可以对不是枚举的类调用此方法：

```
//: enumerated/NonEnum.java

public class NonEnum {
  public static void main(String[] args) {
    Class<Integer> intClass = Integer.class;
    try {
      for(Object en : intClass.getEnumConstants())
        System.out.println(en);
    } catch(Exception e) {
      System.out.println(e);
    }
  }
} /* Output:
java.lang.NullPointerException
*///:~
```

只不过，此时该方法返回**null**，所以当你试图使用其返回的结果时会发生异常。

19.5 实现，而非继承

我们已经知道，所有的**enum**都继承自**java.lang.Enum**类。由于Java不支持多重继承，所以你的**enum**不能再继承其他类：

```
enum NotPossible extends Pet { ... // Won't work
```

然而，在我们创建一个新的**enum**时，可以同时实现一个或多个接口：

```
//: enumerated/cartoons/EnumImplementation.java
// An enum can implement an interface
package enumerated.cartoons;
import java.util.*;
import net.mindview.util.*;
enum CartoonCharacter
implements Generator<CartoonCharacter> {
  SLAPPY, SPANKY, PUNCHY, SILLY, BOUNCY, NUTTY, BOB;
  private Random rand = new Random(47);
  public CartoonCharacter next() {
    return values()[rand.nextInt(values().length)];
  }
}

public class EnumImplementation {
  public static <T> void printNext(Generator<T> rg) {
    System.out.print(rg.next() + ", ");
  }
  public static void main(String[] args) {
    // Choose any instance:
    CartoonCharacter cc = CartoonCharacter.BOB;
    for(int i = 0; i < 10; i++)
      printNext(cc);
  }
} /* Output:
BOB, PUNCHY, BOB, SPANKY, NUTTY, PUNCHY, SLAPPY, NUTTY,
NUTTY, SLAPPY,
*///:~
```

这个结果有点奇怪，不过你必须要有一个**enum**实例才能调用其上的方法。现在，在任何接受**Generator**参数的方法中，例如**printNext()**，都可以使用**CartoonCharacter**。

练习2：(2) 修改上例，编写一个**static next()**方法取代实现**Generater**接口。对比这两种方式，各自有什么优缺点。

19.6 随机选取

就像你在**CartoonCharacter.next()**中看到的那样，本章中的很多示例都需要从**enum**实例中进行随机选择。我们可以利用泛型，从而使得这个工作更一般化，并将其加入到我们的工具库中。

```
//: net/mindview/util/Enums.java
package net.mindview.util;
import java.util.*;

public class Enums {
  private static Random rand = new Random(47);
  public static <T extends Enum<T>> T random(Class<T> ec) {
    return random(ec.getEnumConstants());
  }
  public static <T> T random(T[] values) {
    return values[rand.nextInt(values.length)];
```

```
  }
} ///:~
```

古怪的语法<T extends Enum<T>>表示T是一个enum实例。而将Class<T>作为参数的话，我们就可以利用Class对象得到enum实例的数组了。重载后的random()方法只需使用T[]作为参数，因为它并不会调用Enum上的任何操作，它只需从数组中随机选择一个元素即可。这样，最终的返回类型正是enum的类型。

下面是random()方法的一个简单示例：

```
//: enumerated/RandomTest.java
import net.mindview.util.*;

enum Activity { SITTING, LYING, STANDING, HOPPING,
  RUNNING, DODGING, JUMPING, FALLING, FLYING }

public class RandomTest {
  public static void main(String[] args) {
    for(int i = 0; i < 20; i++)
      System.out.print(Enums.random(Activity.class) + " ");
  }
} /* Output:
STANDING FLYING RUNNING STANDING RUNNING STANDING LYING
DODGING SITTING RUNNING HOPPING HOPPING HOPPING RUNNING
STANDING LYING FALLING RUNNING FLYING LYING
*///:~
```

虽然Enum只是一个相当短小的类，但是在本章中你会发现，它能消除很多重复的代码。重复总会制造麻烦，因此消除重复总是有益处的。

19.7 使用接口组织枚举

无法从enum继承子类有时很令人沮丧。这种需求有时源自我们希望扩展原enum中的元素，有时是因为我们希望使用子类将一个enum中的元素进行分组。 1022

在一个接口的内部，创建实现该接口的枚举，以此将元素进行分组，可以达到将枚举元素分类组织的目的。举例来说，假设你想用enum来表示不同类别的食物，同时还希望每个enum元素仍然保持Food类型。那可以这样实现：

```
//: enumerated/menu/Food.java
// Subcategorization of enums within interfaces.
package enumerated.menu;

public interface Food {
  enum Appetizer implements Food {
    SALAD, SOUP, SPRING_ROLLS;
  }
  enum MainCourse implements Food {
    LASAGNE, BURRITO, PAD_THAI,
    LENTILS, HUMMOUS, VINDALOO;
  }
  enum Dessert implements Food {
    TIRAMISU, GELATO, BLACK_FOREST_CAKE,
    FRUIT, CREME_CARAMEL;
  }
  enum Coffee implements Food {
    BLACK_COFFEE, DECAF_COFFEE, ESPRESSO,
    LATTE, CAPPUCCINO, TEA, HERB_TEA;
  }
} ///:~
```

对于enum而言，实现接口是使其子类化的唯一办法，所以嵌入在Food中的每个enum都实现了Food接口。现在，在下面的程序中，我们可以说“所有东西都是某种类型的Food”：

```
//: enumerated/menu/TypeOfFood.java
package enumerated.menu;
import static enumerated.menu.Food.*;

public class TypeOfFood {
  public static void main(String[] args) {
    Food food = Appetizer.SALAD;
    food = MainCourse.LASAGNE;
    food = Dessert.GELATO;
    food = Coffee.CAPPUCCINO;
  }
} ///:~
```

1023

如果**enum**类型实现了**Food**接口，那么我们就可以将其实例向上转型为**Food**，所以上例中的所有东西都是**Food**。

然而，当你需要与一大堆类型打交道时，接口就不如**enum**好用了。例如，如果你想创建一个“枚举的枚举”，那么可以创建一个新的**enum**，然后用其实例包装**Food**中的每一个**enum**类：

```
//: enumerated/menu/Course.java
package enumerated.menu;
import net.mindview.util.*;

public enum Course {
  APPETIZER(Food.Appetizer.class),
  MAINCOURSE(Food.MainCourse.class),
  DESSERT(Food.Dessert.class),
  COFFEE(Food.Coffee.class);
  private Food[] values;
  private Course(Class<? extends Food> kind) {
    values = kind.getEnumConstants();
  }
  public Food randomSelection() {
    return Enums.random(values);
  }
} ///:~
```

在上面的程序中，每一个**Course**的实例都将其对应的**Class**对象作为构造器的参数。通过**getEnumConstants()**方法，可以从该**Class**对象中取得某个**Food**子类的所有**enum**实例。这些实例在**randomSelection()**中被用到。因此，通过从每一个**Course**实例中随机地选择一个**Food**，我们便能够生成一份菜单：

```
//: enumerated/menu/Meal.java
package enumerated.menu;

public class Meal {
  public static void main(String[] args) {
    for(int i = 0; i < 5; i++) {
      for(Course course : Course.values()) {
        Food food = course.randomSelection();
        System.out.println(food);
      }
      System.out.println("---");
    }
  }
} /* Output:
SPRING_ROLLS
VINDALOO
FRUIT
DECAF_COFFEE
---
SOUP
VINDALOO
FRUIT
```

1024

```
TEA
---
SALAD
BURRITO
FRUIT
TEA
---
SALAD
BURRITO
CREME_CARAMEL
LATTE
---
SOUP
BURRITO
TIRAMISU
ESPRESSO
---
*///:~
```

在这个例子中，我们通过遍历每一个**Course**实例来获得“枚举的枚举”的值。稍后，在**VendingMachine.java**中，我们会看到另一种组织枚举实例的方式，但其也有一些其他的限制。

此外，还有一种更简洁的管理枚举的办法，就是将一个**enum**嵌套在另一个**enum**内。就像这样：

```
//: enumerated/SecurityCategory.java
// More succinct subcategorization of enums.
import net.mindview.util.*;

enum SecurityCategory {
  STOCK(Security.Stock.class), BOND(Security.Bond.class);
  Security[] values;
  SecurityCategory(Class<? extends Security> kind) {
    values = kind.getEnumConstants();
  }
  interface Security {
    enum Stock implements Security { SHORT, LONG, MARGIN }
    enum Bond implements Security { MUNICIPAL, JUNK }
  }
  public Security randomSelection() {
    return Enums.random(values);
  }
  public static void main(String[] args) {
    for(int i = 0; i < 10; i++) {
      SecurityCategory category =
        Enums.random(SecurityCategory.class);
      System.out.println(category + ": " +
        category.randomSelection());
    }
  }
} /* Output:
BOND: MUNICIPAL
BOND: MUNICIPAL
STOCK: MARGIN
STOCK: MARGIN
BOND: JUNK
STOCK: SHORT
STOCK: LONG
STOCK: LONG
BOND: MUNICIPAL
BOND: JUNK
*///:~
```

1025

Security接口的作用是将其所包含的**enum**组合成一个公共类型，这一点是有必要的。然后，**SecurityCategory**才能将**Security**中的**enum**作为其构造器的参数使用，以起到组织的效果。

如果我们将这种方式应用于**Food**的例子，结果应该这样：

```
//: enumerated/menu/Meal2.java
package enumerated.menu;
import net.mindview.util.*;

public enum Meal2 {
  APPETIZER(Food.Appetizer.class),
  MAINCOURSE(Food.MainCourse.class),
  DESSERT(Food.Dessert.class),
  COFFEE(Food.Coffee.class);
  private Food[] values;
  private Meal2(Class<? extends Food> kind) {
    values = kind.getEnumConstants();
  }
  public interface Food {
    enum Appetizer implements Food {
      SALAD, SOUP, SPRING_ROLLS;
    }
    enum MainCourse implements Food {
      LASAGNE, BURRITO, PAD_THAI,
      LENTILS, HUMMOUS, VINDALOO;
    }
    enum Dessert implements Food {
      TIRAMISU, GELATO, BLACK_FOREST_CAKE,
      FRUIT, CREME_CARAMEL;
    }
    enum Coffee implements Food {
      BLACK_COFFEE, DECAF_COFFEE, ESPRESSO,
      LATTE, CAPPUCCINO, TEA, HERB_TEA;
    }
  }
  public Food randomSelection() {
    return Enums.random(values);
  }
  public static void main(String[] args) {
    for(int i = 0; i < 5; i++) {
      for(Meal2 meal : Meal2.values()) {
        Food food = meal.randomSelection();
        System.out.println(food);
      }
      System.out.println("---");
    }
  }
} /* Same output as Meal.java *///:~
```

1026

其实，这仅仅是重新组织了一下代码，不过多数情况下，这种方式使你的代码具有更清晰的结构。

练习3：(1) 向**Course.java**中添加一个新的**Course**，证明它在**Meal.java**中能正确工作。

练习4：(1) 针对**Meal2.java**，重复前一个练习。

练习5：(4) 修改**control/VowelsAndConsonants.java**，使用3个**enum**类型：**VOWEL**，**SOMETIMES_A_VOWEL**，以及**CONSONANT**。其中的**enum**构造器应该可以接受属于不同类别的各种字母。提示：使用可变参数。要记住，可变参数会自动为你创建一个数组。

1027

练习6：(3) 试比较以下两种方式的优缺点：第一：将**Appetizer**、**MainCourse**、**Desert**和**Coffee**嵌入在**Food**内部；第二：将它们实现为单独的**enum**，并各自实现**Food**接口。

19.8 使用EnumSet替代标志

Set是一种集合，只能向其中添加不重复的对象。当然，**enum**也要求其成员都是唯一的，所以**enum**看起来也具有集合的行为。不过，由于不能从**enum**中删除或添加元素，所以它只能算是不太有用的集合。Java SE5引入**EnumSet**，是为了通过**enum**创建一种替代品，以替代传统的

基于int的“位标志”。这种标志可以用来表示某种“开/关”信息，不过，使用这种标志，我们最终操作的只是一些bit，而不是这些bit想要表达的概念，因此很容易写出令人难以理解的代码。

EnumSet的设计充分考虑到了速度因素，因为它必须与非常高效的bit标志相竞争（其操作与**HashSet**相比，非常地快）。就其内部而言，它（可能）就是将一个**long**值作为比特向量，所以**EnumSet**非常快速高效。使用**EnumSet**的优点是，它在说明一个二进制位是否存在时，具有更好的表达能力，并且无需担心性能。

EnumSet中的元素必须来自一个**enum**。下面的**enum**表示在一座大楼中，警报传感器的安放位置：

```
//: enumerated/AlarmPoints.java
package enumerated;
public enum AlarmPoints {
  STAIR1, STAIR2, LOBBY, OFFICE1, OFFICE2, OFFICE3,
  OFFICE4, BATHROOM, UTILITY, KITCHEN
} ///:~
```

然后，我们用**EnumSet**来跟踪报警器的状态：

```
//: enumerated/EnumSets.java
// Operations on EnumSets
package enumerated;
import java.util.*;
import static enumerated.AlarmPoints.*;
import static net.mindview.util.Print.*;
public class EnumSets {
  public static void main(String[] args) {
    EnumSet<AlarmPoints> points =
      EnumSet.noneOf(AlarmPoints.class); // Empty set
    points.add(BATHROOM);
    print(points);
    points.addAll(EnumSet.of(STAIR1, STAIR2, KITCHEN));
    print(points);
    points = EnumSet.allOf(AlarmPoints.class);
    points.removeAll(EnumSet.of(STAIR1, STAIR2, KITCHEN));
    print(points);
    points.removeAll(EnumSet.range(OFFICE1, OFFICE4));
    print(points);
    points = EnumSet.complementOf(points);
    print(points);
  }
} /* Output:
[BATHROOM]
[STAIR1, STAIR2, BATHROOM, KITCHEN]
[LOBBY, OFFICE1, OFFICE2, OFFICE3, OFFICE4, BATHROOM,
UTILITY]
[LOBBY, BATHROOM, UTILITY]
[STAIR1, STAIR2, OFFICE1, OFFICE2, OFFICE3, OFFICE4,
KITCHEN]
*///:~
```

1028

使用**static import**可以简化**enum**常量的使用。**EnumSet**的方法的名字都相当直观，你可以查阅JDK文档找到其完整详细的描述。如果仔细研究了**EnumSet**的文档，你还会发现一个有趣的地方：**of()**方法被重载了很多次，不但为可变数量参数进行了重载，而且为接收2至5个显式的参数的情况都进行了重载。这也从侧面表现了**EnumSet**对性能的关注。因为，其实只使用可变参数已经可以解决整个问题了，但是对比显式的参数，会有一点性能损失。采用现在这种设计，当你只使用2到5个参数调用**of()**方法时，你可以调用对应的重载过的方法（速度稍快一点），而当你使用一个参数或多过5个参数时，你调用的将是使用可变参数的**of()**方法。注意，如果你只使用一个参数，编译器并不会构造可变参数的数组，所以与调用只有一个参数的方法相比，也

就不会有额外的性能损耗。

EnumSet的基础是**long**，一个**long**值有64位，而一个**enum**实例只需一位bit表示其是否存在。也就是说，在不超过一个**long**的表达能力的情况下，你的**EnumSet**可以应用于最多不超过64个元素的**enum**。如果**enum**超过了64个元素会发生什么呢？

1029

```
//: enumerated/BigEnumSet.java
import java.util.*;

public class BigEnumSet {
  enum Big { A0, A1, A2, A3, A4, A5, A6, A7, A8, A9, A10,
    A11, A12, A13, A14, A15, A16, A17, A18, A19, A20, A21,
    A22, A23, A24, A25, A26, A27, A28, A29, A30, A31, A32,
    A33, A34, A35, A36, A37, A38, A39, A40, A41, A42, A43,
    A44, A45, A46, A47, A48, A49, A50, A51, A52, A53, A54,
    A55, A56, A57, A58, A59, A60, A61, A62, A63, A64, A65,
    A66, A67, A68, A69, A70, A71, A72, A73, A74, A75 }
  public static void main(String[] args) {
    EnumSet<Big> bigEnumSet = EnumSet.allOf(Big.class);
    System.out.println(bigEnumSet);
  }
} /* Output:
[A0, A1, A2, A3, A4, A5, A6, A7, A8, A9, A10, A11, A12,
A13, A14, A15, A16, A17, A18, A19, A20, A21, A22, A23, A24,
A25, A26, A27, A28, A29, A30, A31, A32, A33, A34, A35, A36,
A37, A38, A39, A40, A41, A42, A43, A44, A45, A46, A47, A48,
A49, A50, A51, A52, A53, A54, A55, A56, A57, A58, A59, A60,
A61, A62, A63, A64, A65, A66, A67, A68, A69, A70, A71, A72,
A73, A74, A75]
*///:~
```

显然，**EnumSet**可以应用于多过64个元素的**enum**，所以我猜测，**Enum**会在必要的时候增加一个**long**。

练习7：(3) 找到**EnumSet**的源代码，解释其工作原理。

19.9 使用EnumMap

EnumMap是一种特殊的**Map**，它要求其中的键（key）必须来自一个**enum**。由于**enum**本身的限制，所以**EnumMap**在内部可由数组实现。因此**EnumMap**的速度很快，我们可以放心地使用**enum**实例在**EnumMap**中进行查找操作。不过，我们只能将**enum**的实例作为键来调用**put()**方法，其他操作与使用一般的**Map**差不多。

1030

下面的例子演示了命令设计模式的用法。一般来说，命令模式首先需要一个只有单一方法的接口，然后从该接口实现具有各自不同的行为的多个子类。接下来，程序员就可以构造命令对象，并在需要的时候使用它们了：

```
//: enumerated/EnumMaps.java
// Basics of EnumMaps.
package enumerated;
import java.util.*;
import static enumerated.AlarmPoints.*;
import static net.mindview.util.Print.*;

interface Command { void action(); }

public class EnumMaps {
  public static void main(String[] args) {
    EnumMap<AlarmPoints,Command> em =
      new EnumMap<AlarmPoints,Command>(AlarmPoints.class);
    em.put(KITCHEN, new Command() {
      public void action() { print("Kitchen fire!"); }
```

```
    });
    em.put(BATHROOM, new Command() {
      public void action() { print("Bathroom alert!"); }
    });
    for(Map.Entry<AlarmPoints,Command> e : em.entrySet()) {
      printnb(e.getKey() + ": ");
      e.getValue().action();
    }
    try { // If there's no value for a particular key:
      em.get(UTILITY).action();
    } catch(Exception e) {
      print(e);
    }
  }
} /* Output:
BATHROOM: Bathroom alert!
KITCHEN: Kitchen fire!
java.lang.NullPointerException
*///:~
```

与**EnumSet**一样，**enum**实例定义时的次序决定了其在**EnumMap**中的顺序。 1031

main()方法的最后部分说明，**enum**的每个实例作为一个键，总是存在的。但是，如果你没有为这个键调用**put()**方法来存入相应的值的话，其对应的值就是**null**。

与*常量相关的方法*（constant-specific methods将在下一节中介绍）相比，**EnumMap**有一个优点，那**EnumMap**允许程序员改变值对象，而常量相关的方法在编译期就被固定了。

稍后你会看到，在你有多种类型的**enum**，而且它们之间存在互操作的情况下，我们可以用**EnumMap**实现*多路分发*（multiple dispatching）。

19.10 常量相关的方法

Java的**enum**有一个非常有趣的特性，即它允许程序员为**enum**实例编写方法，从而为每个**enum**实例赋予各自不同的行为。要实现常量相关的方法，你需要为**enum**定义一个或多个**abstract**方法，然后为每个**enum**实例实现该抽象方法。参考下面的例子：

```
//: enumerated/ConstantSpecificMethod.java
import java.util.*;
import java.text.*;

public enum ConstantSpecificMethod {
  DATE_TIME {
    String getInfo() {
      return
        DateFormat.getDateInstance().format(new Date());
    }
  },
  CLASSPATH {
    String getInfo() {
      return System.getenv("CLASSPATH");
    }
  },
  VERSION {
    String getInfo() {
      return System.getProperty("java.version");
    }
  };
  abstract String getInfo();
  public static void main(String[] args) {
    for(ConstantSpecificMethod csm : values())
      System.out.println(csm.getInfo());
  }
} /* (Execute to see output) *///:~
```

1032

通过相应的**enum**实例，我们可以调用其上的方法。这通常也称为*表驱动的代码*（table-driven code，请注意它与前面提到的命令模式的相似之处）。

在面向对象的程序设计中，不同的行为与不同的类关联。而通过常量相关的方法，每个**enum**实例可以具备自己独特的行为，这似乎说明每个**enum**实例就像一个独特的类。在上面的例子中，**enum**实例似乎被当作其“超类”**ConstantSpecificMethod**来使用，在调用**getInfo()**方法时，体现出多态的行为。

然而，**enum**实例与类的相似之处也仅限于此了。我们并不能真的将**enum**实例作为一个类型来使用：

```
//: enumerated/NotClasses.java
// {Exec: javap -c LikeClasses}
import static net.mindview.util.Print.*;

enum LikeClasses {
  WINKEN { void behavior() { print("Behavior1"); } },
  BLINKEN { void behavior() { print("Behavior2"); } },
  NOD { void behavior() { print("Behavior3"); } };
  abstract void behavior();
}

public class NotClasses {
  // void f1(LikeClasses.WINKEN instance) {} // Nope
} /* Output:
Compiled from "NotClasses.java"
abstract class LikeClasses extends java.lang.Enum{
public static final LikeClasses WINKEN;

public static final LikeClasses BLINKEN;

public static final LikeClasses NOD;
...
*///:~
```

在方法**f1()**中，编译器不允许我们将一个**enum**实例当作**class**类型。如果我们分析一下编译器生成的代码，就知道这种行为也是很正常的。因为每个**enum**元素都是一个**LikeClasses**类型的**static final**实例。

1033

同时，由于它们是**static**实例，无法访问外部类的非**static**元素或方法，所以对于内部的**enum**的实例而言，其行为与一般的内部类并不相同。

再看一个更有趣的关于洗车的例子。每个顾客在洗车时，都有一个选择菜单，每个选择对应一个不同的动作。可以将一个常量相关的方法关联到一个选择上，再使用一个**EnumSet**来保存客户的选择：

```
//: enumerated/CarWash.java
import java.util.*;
import static net.mindview.util.Print.*;

public class CarWash {
  public enum Cycle {
    UNDERBODY {
      void action() { print("Spraying the underbody"); }
    },
    WHEELWASH {
      void action() { print("Washing the wheels"); }
    },
    PREWASH {
      void action() { print("Loosening the dirt"); }
    },
    BASIC {
      void action() { print("The basic wash"); }
```

```
    },
    HOTWAX {
      void action() { print("Applying hot wax"); }
    },
    RINSE {
      void action() { print("Rinsing"); }
    },
    BLOWDRY {
      void action() { print("Blowing dry"); }
    };
    abstract void action();
  }
  EnumSet<Cycle> cycles =
    EnumSet.of(Cycle.BASIC, Cycle.RINSE);
  public void add(Cycle cycle) { cycles.add(cycle); }
  public void washCar() {
    for(Cycle c : cycles)
      c.action();
  }
  public String toString() { return cycles.toString(); }
  public static void main(String[] args) {
    CarWash wash = new CarWash();
    print(wash);
    wash.washCar();
    // Order of addition is unimportant:
    wash.add(Cycle.BLOWDRY);
    wash.add(Cycle.BLOWDRY); // Duplicates ignored
    wash.add(Cycle.RINSE);
    wash.add(Cycle.HOTWAX);
    print(wash);
    wash.washCar();
  }
} /* Output:
[BASIC, RINSE]
The basic wash
Rinsing
[BASIC, HOTWAX, RINSE, BLOWDRY]
The basic wash
Applying hot wax
Rinsing
Blowing dry
*///:~
```

1034

与使用匿名内部类相比较，定义常量相关方法的语法更高效、简洁。

这个例子也展示了**EnumSet**了一些特性。因为它是一个集合，所以对于同一个元素而言，只能出现一次，因此对同一个参数重复地调用**add()**方法会被忽略掉（这是正确的行为，因为一个bit位开关只能“打开”一次）。同样地，向**EnumSet**添加**enum**实例的顺序并不重要，因为其输出的次序决定于**enum**实例定义时的次序。

除了实现**abstract**方法以外，程序员是否可以覆盖常量相关的方法呢？答案是肯定的，参考下面的程序：

```
//: enumerated/OverrideConstantSpecific.java
import static net.mindview.util.Print.*;

public enum OverrideConstantSpecific {
  NUT, BOLT,
  WASHER {
    void f() { print("Overridden method"); }
  };
  void f() { print("default behavior"); }
  public static void main(String[] args) {
    for(OverrideConstantSpecific ocs : values()) {
      printnb(ocs + ": ");
      ocs.f();
```

1035

```
    }
  }
} /* Output:
NUT: default behavior
BOLT: default behavior
WASHER: Overridden method
*///:~
```

虽然**enum**有某些限制，但是一般而言，我们还是可以将其看作是类。

19.10.1 使用enum的职责链

在职责链（Chain of Responsibility）设计模式中，程序员以多种不同的方式来解决一个问题，然后将它们链接在一起。当一个请求到来时，它遍历这个链，直到链中的某个解决方案能够处理该请求。

通过常量相关的方法，我们可以很容易地实现一个简单的职责链。我们以一个邮局的模型为例。邮局需要以尽可能通用的方式来处理每一封邮件，并且要不断尝试处理邮件，直到该邮件最终被确定为一封死信。其中的每一次尝试可以看作为一个策略（也是一个设计模式），而完整的处理方式列表就是一个职责链。

我们先来描述一下邮件。邮件的每个关键特征都可以用**enum**来表示。程序将随机地生成**Mail**对象，如果要减小一封邮件的**GeneralDelivery**为**YES**的概率，那最简单的方法就是多创建几个不是**YES**的**enum**实例，所以**enum**的定义看起来有点古怪。

我们看到**Mail**中有一个**randomMail()**方法，它负责随机地创建用于测试的邮件。而**generator()**方法生成一个**Iterable**对象，该对象在你调用**next()**方法时，在其内部使用**randomMail()**来创建**Mail**对象。这样的结构使程序员可以通过调用**Mail.generator()**方法，很容易地构造出一个foreach循环：

1036

```
//: enumerated/PostOffice.java
// Modeling a post office.
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

class Mail {
  // The NO's lower the probability of random selection:
  enum GeneralDelivery {YES,NO1,NO2,NO3,NO4,NO5}
  enum Scannability {UNSCANNABLE,YES1,YES2,YES3,YES4}
  enum Readability {ILLEGIBLE,YES1,YES2,YES3,YES4}
  enum Address {INCORRECT,OK1,OK2,OK3,OK4,OK5,OK6}
  enum ReturnAddress {MISSING,OK1,OK2,OK3,OK4,OK5}
  GeneralDelivery generalDelivery;
  Scannability scannability;
  Readability readability;
  Address address;
  ReturnAddress returnAddress;
  static long counter = 0;
  long id = counter++;
  public String toString() { return "Mail " + id; }
  public String details() {
    return toString() +
      ", General Delivery: " + generalDelivery +
      ", Address Scanability: " + scannability +
      ", Address Readability: " + readability +
      ", Address Address: " + address +
      ", Return address: " + returnAddress;
  }
  // Generate test Mail:
  public static Mail randomMail() {
    Mail m = new Mail();
    m.generalDelivery= Enums.random(GeneralDelivery.class);
```

```
    m.scannability = Enums.random(Scannability.class);
    m.readability = Enums.random(Readability.class);
    m.address = Enums.random(Address.class);
    m.returnAddress = Enums.random(ReturnAddress.class);
    return m;
  }
  public static Iterable<Mail> generator(final int count) {
    return new Iterable<Mail>() {
      int n = count;
      public Iterator<Mail> iterator() {
        return new Iterator<Mail>() {
          public boolean hasNext() { return n-- > 0; }
          public Mail next() { return randomMail(); }
          public void remove() { // Not implemented
            throw new UnsupportedOperationException();
          }
        };
      }
    };
  }
}

public class PostOffice {
  enum MailHandler {
    GENERAL_DELIVERY {
      boolean handle(Mail m) {
        switch(m.generalDelivery) {
          case YES:
            print("Using general delivery for " + m);
            return true;
          default: return false;
        }
      }
    },
    MACHINE_SCAN {
      boolean handle(Mail m) {
        switch(m.scannability) {
          case UNSCANNABLE: return false;
          default:
            switch(m.address) {
              case INCORRECT: return false;
              default:
                print("Delivering "+ m + " automatically");
                return true;
            }
        }
      }
    },
    VISUAL_INSPECTION {
      boolean handle(Mail m) {
        switch(m.readability) {
          case ILLEGIBLE: return false;
          default:
            switch(m.address) {
              case INCORRECT: return false;
              default:
                print("Delivering " + m + " normally");
                return true;
            }
        }
      }
    },
    RETURN_TO_SENDER {
      boolean handle(Mail m) {
        switch(m.returnAddress) {
          case MISSING: return false;
          default:
```

1037

1038

```
            print("Returning " + m + " to sender");
            return true;
          }
        }
      };
      abstract boolean handle(Mail m);
    }
    static void handle(Mail m) {
      for(MailHandler handler : MailHandler.values())
        if(handler.handle(m))
          return;
      print(m + " is a dead letter");
    }
    public static void main(String[] args) {
      for(Mail mail : Mail.generator(10)) {
        print(mail.details());
        handle(mail);
        print("*****");
      }
    }
  } /* Output:
  Mail 0, General Delivery: NO2, Address Scanability:
  UNSCANNABLE, Address Readability: YES3, Address Address:
  OK1, Return address: OK1
  Delivering Mail 0 normally
  *****
  Mail 1, General Delivery: NO5, Address Scanability: YES3,
  Address Readability: ILLEGIBLE, Address Address: OK5,
  Return address: OK1
  Delivering Mail 1 automatically
  *****
  Mail 2, General Delivery: YES, Address Scanability: YES3,
  Address Readability: YES1, Address Address: OK1, Return
  address: OK5
  Using general delivery for Mail 2
  *****
  Mail 3, General Delivery: NO4, Address Scanability: YES3,
  Address Readability: YES1, Address Address: INCORRECT,
  Return address: OK4
  Returning Mail 3 to sender
  *****
  Mail 4, General Delivery: NO4, Address Scanability:
  UNSCANNABLE, Address Readability: YES1, Address Address:
  INCORRECT, Return address: OK2
  Returning Mail 4 to sender
  *****
  Mail 5, General Delivery: NO3, Address Scanability: YES1,
  Address Readability: ILLEGIBLE, Address Address: OK4,
  Return address: OK2
  Delivering Mail 5 automatically
  *****
  Mail 6, General Delivery: YES, Address Scanability: YES4,
  Address Readability: ILLEGIBLE, Address Address: OK4,
  Return address: OK4
  Using general delivery for Mail 6
  *****
  Mail 7, General Delivery: YES, Address Scanability: YES3,
  Address Readability: YES4, Address Address: OK2, Return
  address: MISSING
  Using general delivery for Mail 7
  *****
  Mail 8, General Delivery: NO3, Address Scanability: YES1,
  Address Readability: YES3, Address Address: INCORRECT,
  Return address: MISSING
  Mail 8 is a dead letter
  *****
  Mail 9, General Delivery: NO1, Address Scanability:
```

1039

```
UNSCANNABLE, Address Readability: YES2, Address Address:
OK1, Return address: OK4
Delivering Mail 9 normally
*****
*///:~
```

职责链由**enum MailHandler**实现，而**enum**定义的次序决定了各个解决策略在应用时的次序。对每一封邮件，都要按此顺序尝试每个解决策略，直到其中一个能够成功地处理该邮件，如果所有的策略都失败了，那么该邮件将被判定为一封死信。 1040

练习8：(6) 修改**PostOffice.java**，使其能够转发邮件。

练习9：(5) 修改**class PostOffice**，使其能够使用**EnumMap**。

作业[⊖]：专用程序设计语言，例如Prolog，使用反向链来解决类似的问题。试用**PostOffice.java**做一个例子，研究一下这些语言，用其编写一个扩展性更好的程序，使程序员可以很容易地向系统中添加新的“规则”。

19.10.2 使用enum的状态机

枚举类型非常适合用来创建状态机。一个状态机可以具有有限个特定的状态，它通常根据输入，从一个状态转移到下一个状态，不过也可能存在瞬时状态（transient states），而一旦任务执行结束，状态机就会立刻离开瞬时状态。

每个状态都具有某些可接受的输入，不同的输入会使状态机从当前状态转移到不同的新状态。由于**enum**对其实例有严格限制，非常适合用来表现不同的状态和输入。一般而言，每个状态都具有一些相关的输出。

自动售货机是一个很好的状态机的例子。首先，我们用一个**enum**定义各种输入：

```
//: enumerated/Input.java
package enumerated;
import java.util.*;

public enum Input {
  NICKEL(5), DIME(10), QUARTER(25), DOLLAR(100),
  TOOTHPASTE(200), CHIPS(75), SODA(100), SOAP(50),
  ABORT_TRANSACTION {
    public int amount() { // Disallow
      throw new RuntimeException("ABORT.amount()");
    }
  },
  STOP { // This must be the last instance.
    public int amount() { // Disallow
      throw new RuntimeException("SHUT_DOWN.amount()");
    }
  };
  int value; // In cents
  Input(int value) { this.value = value; }
  Input() {}
  int amount() { return value; }; // In cents
  static Random rand = new Random(47);
  public static Input randomSelection() {
    // Don't include STOP:
    return values()[rand.nextInt(values().length - 1)];
  }
} ///:~
```

1041

注意，除了两个特殊的**Input**实例之外，其他的**Input**都有相应的价格，因此在接口中定义了**amount()**方法。然而，对那两个特殊**Input**实例而言，调用**amount()**方法并不合适，所以如果程序员调用它们的**amount()**方法就会有异常抛出（在接口内定义了一个方法，然后在你调用该

⊖ 作业，我建议读者将其作为课程大作业。解答指南中不包含此类作业的解决方案。

方法的某个实现时就会抛出异常)。这似乎有点奇怪，但由于enum的限制，我们不得不采用这种方式。

VendingMachine对输入的第一个反应是将其归类为Category enum中的某一个enum实例，这可以通过switch实现。下面的例子演示了enum是如何使代码变得更加清晰且易于管理的：

```
//: enumerated/VendingMachine.java
// {Args: VendingMachineInput.txt}
package enumerated;
import java.util.*;
import net.mindview.util.*;
import static enumerated.Input.*;
import static net.mindview.util.Print.*;

enum Category {
  MONEY(NICKEL, DIME, QUARTER, DOLLAR),
  ITEM_SELECTION(TOOTHPASTE, CHIPS, SODA, SOAP),
  QUIT_TRANSACTION(ABORT_TRANSACTION),
  SHUT_DOWN(STOP);
  private Input[] values;
  Category(Input... types) { values = types; }
  private static EnumMap<Input,Category> categories =
    new EnumMap<Input,Category>(Input.class);
  static {
    for(Category c : Category.class.getEnumConstants())
      for(Input type : c.values)
        categories.put(type, c);
  }
  public static Category categorize(Input input) {
    return categories.get(input);
  }
}

public class VendingMachine {
  private static State state = State.RESTING;
  private static int amount = 0;
  private static Input selection = null;
  enum StateDuration { TRANSIENT } // Tagging enum
  enum State {
    RESTING {
      void next(Input input) {
        switch(Category.categorize(input)) {
          case MONEY:
            amount += input.amount();
            state = ADDING_MONEY;
            break;
          case SHUT_DOWN:
            state = TERMINAL;
          default:
        }
      }
    },
    ADDING_MONEY {
      void next(Input input) {
        switch(Category.categorize(input)) {
          case MONEY:
            amount += input.amount();
            break;
          case ITEM_SELECTION:
            selection = input;
            if(amount < selection.amount())
              print("Insufficient money for " + selection);
            else state = DISPENSING;
            break;
          case QUIT_TRANSACTION:
            state = GIVING_CHANGE;
```

1042

1043

```
            break;
          case SHUT_DOWN:
            state = TERMINAL;
          default:
        }
      }
    },
    DISPENSING(StateDuration.TRANSIENT) {
      void next() {
        print("here is your " + selection);
        amount -= selection.amount();
        state = GIVING_CHANGE;
      }
    },
    GIVING_CHANGE(StateDuration.TRANSIENT) {
      void next() {
        if(amount > 0) {
          print("Your change: " + amount);
          amount = 0;
        }
        state = RESTING;
      }
    },
    TERMINAL { void output() { print("Halted"); } };
    private boolean isTransient = false;
    State() {}
    State(StateDuration trans) { isTransient = true; }
    void next(Input input) {
      throw new RuntimeException("Only call " +
        "next(Input input) for non-transient states");
    }
    void next() {
      throw new RuntimeException("Only call next() for " +
        "StateDuration.TRANSIENT states");
    }
    void output() { print(amount); }
  }
  static void run(Generator<Input> gen) {
    while(state != State.TERMINAL) {
      state.next(gen.next());
      while(state.isTransient)
        state.next();
      state.output();
    }
  }
  public static void main(String[] args) {
    Generator<Input> gen = new RandomInputGenerator();
    if(args.length == 1)
      gen = new FileInputGenerator(args[0]);
    run(gen);
  }
}

// For a basic sanity check:
class RandomInputGenerator implements Generator<Input> {
  public Input next() { return Input.randomSelection(); }
}

// Create Inputs from a file of ';'-separated strings:
class FileInputGenerator implements Generator<Input> {
  private Iterator<String> input;
  public FileInputGenerator(String fileName) {
    input = new TextFile(fileName, ";").iterator();
  }
  public Input next() {
    if(!input.hasNext())
      return null;
```

1044

```
    return Enum.valueOf(Input.class, input.next().trim());
  }
} /* Output:
25
50
75
here is your CHIPS
0
100
200
here is your TOOTHPASTE
0
25
35
Your change: 35
0
25
35
Insufficient money for SODA
35
60
70
75
Insufficient money for SODA
75
Your change: 75
0
Halted
*///:~
```

1045

由于用**switch**语句从**enum**实例中进行选择是最常见的一种方式（请注意，为了使**enum**在**switch**语句中的使用变得简单，我们是需要付出其他代价的），所以，我们经常遇到这样的问题：将多个**enum**进行分类时，“我们希望在什么**enum**中使用**switch**语句？”我们通过**VendingMachine**的例子来研究一下这个问题。对于每一个**State**，我们都需要在输入动作的基本分类中进行查找：用户塞入钞票，选择了某个货物，操作被取消，以及机器停止。然而，在这些基本分类之下，我们又可以塞入不同类型的钞票，可以选择不同的货物。**Category enum**将不同类型的**Input**进行分组，因而，可以使用**categorize()**方法为**switch**语句生成恰当的**Cateroy**实例。并且，该方法使用的**EnumMap**确保了在其中进行查询时的效率与安全。

如果读者仔细研究**VendingMachine**类，就会发现每种状态的不同之处，以及对于输入的不同响应，其中还有两个瞬时状态。在**run()**方法中，状态机等待着下一个**Input**，并一直在各个状态中移动，直到它不再处于瞬时状态。

通过两种不同的**Generator**对象，我们可以用两种方式来测试**VendingMachine**。首先是**RandomInputGenerator**，它会不停地生成各种输入，当然，除了**SHUT_DOWN**之外。通过长时间地运行**RandomInputGenerator**，可以起到健全测试（sanity test）的作用，能够确保该状态机不会进入一个错误状态。另一个是**FileInputGenerator**，使用文件以文本的方式来描述输入，然后将它们转换成**enum**实例，并创建对应的**Input**对象。上面的程序使用的正是如下的文本文件：

```
//:! enumerated/VendingMachineInput.txt
QUARTER; QUARTER; QUARTER; CHIPS;
DOLLAR; DOLLAR; TOOTHPASTE;
QUARTER; DIME; ABORT_TRANSACTION;
QUARTER; DIME; SODA;
QUARTER; DIME; NICKEL; SODA;
ABORT_TRANSACTION;
STOP;
///:~
```

1046

这种设计有一个缺陷，它要求**enum State**实例访问的**VendingMachine**属性必须声明为**static**，这意味着，你只能有一个**VendingMachine**实例。不过如果我们思考一下实际的（嵌入式Java）应用，这也许并不是一个大问题，因为在一台机器上，我们可能只有一个应用程序。

练习10：(7) 修改**class VendingMachine**，在其中使用**EnumMap**，使其同时可以存在多个**VendingMachine**实例。

练习11：(7) 如果是一个真的自动售货机，我们希望能够很容易地添加或改变售卖货品的种类，因此，用**enum**来表现**Input**时的缺陷使其并不实用（我们知道**enum**对实例有着特殊的限制，一旦声明结束就不能有任何改动）。修改**VendingMachine**，用一个**class**来表现售卖的货品，并基于一个文本文件，使用**ArrayList**来初始化它们（可以使用**net.mindview.util.TextFile**）。

作业[⊖]：设计一个自动贩卖机，使其具备能够很容易地应用于所有国家的国际化的能力。

19.11 多路分发

当你要处理多种交互类型时，程序可能会变得相当杂乱。举例来说，如果一个系统要分析和执行数学表达式。我们可能会声明**Number.plus(Number)**、**Number.multiple(Number)**等等，其中**Number**是各种数字对象的超类。然而，当你声明**a.plus(b)**时，你并不知道**a**或**b**的确切类型，那你如何能让它们正确地交互呢？

你可能从未思考过这个问题的答案。Java只支持*单路分发*。也就是说，如果要执行的操作包含了不止一个类型未知的对象时，那么Java的动态绑定机制只能处理其中一个的类型。这就无法解决我们上面提到的问题。所以，你必须自己来判定其他的类型，从而实现自己的动态绑定行为。 1047

解决上面问题的办法就是*多路分发*（在那个例子中，只有两个分发，一般称之为*两路分发*）。多态只能发生在方法调用时，所以，如果你想使用两路分发，那么就必须有两个方法调用：第一个方法调用决定第一个未知类型，第二个方法调用决定第二个未知的类型。要利用多路分发，程序员必须为每一个类型提供一个实际的方法调用，如果你要处理两个不同的类型体系，就需要为每个类型体系执行一个方法调用。一般而言，程序员需要有设定好的某种配置，以便一个方法调用能够引出更多的方法调用，从而能够在这个过程中处理多种类型。为了达到这种效果，我们需要与多个方法一同工作：因为每个分发都需要一个方法调用。在下面的例子中（实现了“石头、剪刀、布”游戏，也称为RoShamBo）对应的方法是**compete()**和**eval()**，二者都是同一个类型的成员，它们可以产生三种Outcome实例中的一个作为结果[⊜]：

```
//: enumerated/Outcome.java
package enumerated;
public enum Outcome { WIN, LOSE, DRAW } ///:~
```

```
//: enumerated/RoShamBo1.java
// Demonstration of multiple dispatching.
package enumerated;
import java.util.*;
import static enumerated.Outcome.*;

interface Item {
  Outcome compete(Item it);
  Outcome eval(Paper p);
  Outcome eval(Scissors s);
```

⊖ 作业，我建议读者将其作为课程大作业。解答指南中不包含此类作业的解决方案。

⊜ 不记得是在哪位作者的书中读到的，总之这个例子已经存在很多年了。你可以在www.MindView.net上找到它们的C++和Java的版本（在《Thinking in Patterns》中）。

```
  Outcome eval(Rock r);
}

class Paper implements Item {
  public Outcome compete(Item it) { return it.eval(this); }
  public Outcome eval(Paper p) { return DRAW; }
  public Outcome eval(Scissors s) { return WIN; }
  public Outcome eval(Rock r) { return LOSE; }
  public String toString() { return "Paper"; }
}

class Scissors implements Item {
  public Outcome compete(Item it) { return it.eval(this); }
  public Outcome eval(Paper p) { return LOSE; }
  public Outcome eval(Scissors s) { return DRAW; }
  public Outcome eval(Rock r) { return WIN; }
  public String toString() { return "Scissors"; }
}

class Rock implements Item {
  public Outcome compete(Item it) { return it.eval(this); }
  public Outcome eval(Paper p) { return WIN; }
  public Outcome eval(Scissors s) { return LOSE; }
  public Outcome eval(Rock r) { return DRAW; }
  public String toString() { return "Rock"; }
}

public class RoShamBo1 {
  static final int SIZE = 20;
  private static Random rand = new Random(47);
  public static Item newItem() {
    switch(rand.nextInt(3)) {
      default:
      case 0: return new Scissors();
      case 1: return new Paper();
      case 2: return new Rock();
    }
  }
  public static void match(Item a, Item b) {
    System.out.println(
      a + " vs. " + b + ": " +  a.compete(b));
  }
  public static void main(String[] args) {
    for(int i = 0; i < SIZE; i++)
      match(newItem(), newItem());
  }
} /* Output:
Rock vs. Rock: DRAW
Paper vs. Rock: WIN
Paper vs. Rock: WIN
Paper vs. Rock: WIN
Scissors vs. Paper: WIN
Scissors vs. Scissors: DRAW
Scissors vs. Paper: WIN
Rock vs. Paper: LOSE
Paper vs. Paper: DRAW
Rock vs. Paper: LOSE
Paper vs. Scissors: LOSE
Paper vs. Scissors: LOSE
Rock vs. Scissors: WIN
Rock vs. Paper: LOSE
Paper vs. Rock: WIN
Scissors vs. Paper: WIN
Paper vs. Scissors: LOSE
Paper vs. Scissors: LOSE
Paper vs. Scissors: LOSE
Paper vs. Scissors: LOSE
*///:~
```

1048
1049

Item是这几种类型的接口，将会被用作多路分发。**RoShamBo1.match()**有两个**Item**参数，通过调用**Item.compete()**方法开始两路分发。要判定**a**的类型，分发机制会在**a**的实际类型的**compete()**内部起到分发的作用。**compete()**方法通过调用**eval()**来为另一个类型实现第二次分法。将自身（**this**）作为参数调用**eval()**，能够调用重载过的**eval()**方法，这能够保留第一次分发的类型信息。当第二次分发完成时，你就能够知道两个**Item**对象的具体类型了。

要配置好多路分发需要很多的工序，不过要记住，它的好处在于方法调用时的优雅的语法，这避免了在一个方法中判定多个对象的类型的丑陋代码，你只需说，“嘿，你们两个，我不在乎你们是什么类型，请你们自己交流！”不过，在使用多路分发前，请先明确，这种优雅的代码对你确实有重要的意义。

19.11.1 使用enum分发

直接将**RoShamBo1.java**翻译为基于**enum**的版本是有问题的，因为**enum**实例不是类型，不能将**enum**实例作为参数的类型，所以无法重载**eval()**方法。不过，还有很多方式可以实现多路分发，并从**enum**中获益。 1050

一种方式是使用构造器来初始化每个**enum**实例，并以“一组”结果作为参数。这二者放在一块，形成了类似查询表的结构：

```
//: enumerated/RoShamBo2.java
// Switching one enum on another.
package enumerated;
import static enumerated.Outcome.*;

public enum RoShamBo2 implements Competitor<RoShamBo2> {
  PAPER(DRAW, LOSE, WIN),
  SCISSORS(WIN, DRAW, LOSE),
  ROCK(LOSE, WIN, DRAW);
  private Outcome vPAPER, vSCISSORS, vROCK;
  RoShamBo2(Outcome paper,Outcome scissors,Outcome rock) {
    this.vPAPER = paper;
    this.vSCISSORS = scissors;
    this.vROCK = rock;
  }
  public Outcome compete(RoShamBo2 it) {
    switch(it) {
      default:
      case PAPER: return vPAPER;
      case SCISSORS: return vSCISSORS;
      case ROCK: return vROCK;
    }
  }
  public static void main(String[] args) {
    RoShamBo.play(RoShamBo2.class, 20);
  }
} /* Output:
ROCK vs. ROCK: DRAW
SCISSORS vs. ROCK: LOSE
SCISSORS vs. ROCK: LOSE
SCISSORS vs. ROCK: LOSE
PAPER vs. SCISSORS: LOSE
PAPER vs. PAPER: DRAW
PAPER vs. SCISSORS: LOSE
ROCK vs. SCISSORS: WIN
SCISSORS vs. SCISSORS: DRAW
ROCK vs. SCISSORS: WIN
SCISSORS vs. PAPER: WIN
SCISSORS vs. PAPER: WIN
ROCK vs. PAPER: LOSE
ROCK vs. SCISSORS: WIN
SCISSORS vs. ROCK: LOSE
```

1051

```
PAPER vs. SCISSORS: LOSE
SCISSORS vs. PAPER: WIN
SCISSORS vs. PAPER: WIN
SCISSORS vs. PAPER: WIN
SCISSORS vs. PAPER: WIN
*///:~
```

在**compete()**方法中，一旦两种类型都被确定了，那么唯一的操作就是返回结果**Outcome**。然而，你可能还需要调用其他的方法，（例如）甚至是调用在构造器中指定的某个命令对象上的方法。

RoShamBo2.java比之前的例子短小得多，而且更直接，更易于理解。注意，我们仍然是使用两路分发来判定两个对象的类型。在**RoShamBo1.java**中，两次分发都是通过实际的方法调用实现，而在这个例子中，只有第一次分发是实际的方法调用。第二个分发使用的是**switch**，不过这样做是安全的，因为**enum**限制了**switch**语句的选择分支。

在代码中，**enum**被单独抽取出来，因此它可以应用在其他例子中。首先，**Competitor**接口定义了一种类型，该类型的对象可以与另一个**Competitor**相竞争：

```
//: enumerated/Competitor.java
// Switching one enum on another.
package enumerated;

public interface Competitor<T extends Competitor<T>> {
  Outcome compete(T competitor);
} ///:~
```

然后，我们定义两个**static**方法（**static**可以避免显式地指明参数类型）。第一个是**match()**方法，它会为一个**Competitor**对象调用**compete()**方法，并与另一个**Competitor**对象作比较。在这个例子中，我们看到，**match()**方法的参数需要是**Competitor<T>**类型。但是在**play()**方法中，类型参数必须同时是**Enum<T>**类型（因为它将在**Enums.random()**中使用）和**Competitor<T>**类型（因为它将被传递给**match()**方法）：

1052

```
//: enumerated/RoShamBo.java
// Common tools for RoShamBo examples.
package enumerated;
import net.mindview.util.*;

public class RoShamBo {
  public static <T extends Competitor<T>>
  void match(T a, T b) {
    System.out.println(
      a + " vs. " + b + ": " +  a.compete(b));
  }
  public static <T extends Enum<T> & Competitor<T>>
  void play(Class<T> rsbClass, int size) {
    for(int i = 0; i < size; i++)
      match(
        Enums.random(rsbClass),Enums.random(rsbClass));
  }
} ///:~
```

play()方法没有将类型参数**T**作为返回值类型，因此，似乎我们应该在**Class<T>**中使用通配符来代替上面的参数声明。然而，通配符不能扩展多个基类，所以我们必须采用以上的表达式。

19.11.2 使用常量相关的方法

常量相关的方法允许我们为每个**enum**实例提供方法的不同实现，这使得常量相关的方法似乎是实现多路分发的完美解决方案。不过，通过这种方式，**enum**实例虽然可以具有不同的行为，但它们仍然不是类型，不能将其作为方法签名中的参数类型来使用。最好的办法是将**enum**用在

switch语句中，见下例：

```
//: enumerated/RoShamBo3.java
// Using constant-specific methods.
package enumerated;
import static enumerated.Outcome.*;

public enum RoShamBo3 implements Competitor<RoShamBo3> {
  PAPER {
    public Outcome compete(RoShamBo3 it) {
      switch(it) {
        default: // To placate the compiler
        case PAPER: return DRAW;
        case SCISSORS: return LOSE;
        case ROCK: return WIN;
      }
    }
  },
  SCISSORS {
    public Outcome compete(RoShamBo3 it) {
      switch(it) {
        default:
        case PAPER: return WIN;
        case SCISSORS: return DRAW;
        case ROCK: return LOSE;
      }
    }
  },
  ROCK {
    public Outcome compete(RoShamBo3 it) {
      switch(it) {
        default:
        case PAPER: return LOSE;
        case SCISSORS: return WIN;
        case ROCK: return DRAW;
      }
    }
  };
  public abstract Outcome compete(RoShamBo3 it);
  public static void main(String[] args) {
    RoShamBo.play(RoShamBo3.class, 20);
  }
} /* Same output as RoShamBo2.java *///:~
```

虽然这种方式可以工作，但是却不甚合理，如果采用**RoShamBo2.java**的解决方案，那么在添加一个新的类型时，只需更少的代码，而且也更直接。

然而，**RoShamBo3.java**还可以压缩简化一下：

```
//: enumerated/RoShamBo4.java
package enumerated;

public enum RoShamBo4 implements Competitor<RoShamBo4> {
  ROCK {
    public Outcome compete(RoShamBo4 opponent) {
      return compete(SCISSORS, opponent);
    }
  },
  SCISSORS {
    public Outcome compete(RoShamBo4 opponent) {
      return compete(PAPER, opponent);
    }
  },
  PAPER {
    public Outcome compete(RoShamBo4 opponent) {
      return compete(ROCK, opponent);
    }
```

1053
1054

```
  };
  Outcome compete(RoShamBo4 loser, RoShamBo4 opponent) {
    return ((opponent == this) ? Outcome.DRAW
        : ((opponent == loser) ? Outcome.WIN
                               : Outcome.LOSE));
  }
  public static void main(String[] args) {
    RoShamBo.play(RoShamBo4.class, 20);
  }
} /* Same output as RoShamBo2.java *///:~
```

其中，具有两个参数的**compete()**方法执行第二个分发，该方法执行一系列的比较，其行为类似**switch**语句。这个版本的程序更简短，不过却比较难理解。对于一个大型系统而言，难以理解的代码将导致整个系统不够健壮。

19.11.3 使用EnumMap分发

使用**EnumMap**能够实现“真正的”两路分发。**EnumMap**是为**enum**专门设计的一种性能非常好的特殊**Map**。由于我们的目的是摸索出两种未知的类型，所以可以用一个**EnumMap**的**EnumMap**来实现两路分发：

```
//: enumerated/RoShamBo5.java
// Multiple dispatching using an EnumMap of EnumMaps.
package enumerated;
import java.util.*;
import static enumerated.Outcome.*;

enum RoShamBo5 implements Competitor<RoShamBo5> {
  PAPER, SCISSORS, ROCK;
  static EnumMap<RoShamBo5,EnumMap<RoShamBo5,Outcome>>
    table = new EnumMap<RoShamBo5,
      EnumMap<RoShamBo5,Outcome>>(RoShamBo5.class);
  static {
    for(RoShamBo5 it : RoShamBo5.values())
      table.put(it,
        new EnumMap<RoShamBo5,Outcome>(RoShamBo5.class));
    initRow(PAPER, DRAW, LOSE, WIN);
    initRow(SCISSORS, WIN, DRAW, LOSE);
    initRow(ROCK, LOSE, WIN, DRAW);
  }
  static void initRow(RoShamBo5 it,
    Outcome vPAPER, Outcome vSCISSORS, Outcome vROCK) {
    EnumMap<RoShamBo5,Outcome> row =
      RoShamBo5.table.get(it);
    row.put(RoShamBo5.PAPER, vPAPER);
    row.put(RoShamBo5.SCISSORS, vSCISSORS);
    row.put(RoShamBo5.ROCK, vROCK);
  }
  public Outcome compete(RoShamBo5 it) {
    return table.get(this).get(it);
  }
  public static void main(String[] args) {
    RoShamBo.play(RoShamBo5.class, 20);
  }
} /* Same output as RoShamBo2.java *///:~
```

1055

该程序在一个**static**子句中初始化**EnumMap**对象，具体见表格似的**initRow()**方法调用。请注意**compete()**方法，您可以看到，在一行语句中发生了两次分发。

19.11.4 使用二维数组

我们还可以进一步简化实现两路分发的解决方案。我们注意到，每个**enum**实例都有一个固定的值（基于其声明的次序），并且可以通过**ordinal()**方法取得该值。因此我们可以使用二维数组，将竞争者映射到竞争结果。采用这种方式能够获得最简洁、最直接的解决方案（很可能也

是最快速的，虽然我们知道**EnumMap**内部其实也是使用数组实现的）。

```
//: enumerated/RoShamBo6.java
// Enums using "tables" instead of multiple dispatch.
package enumerated;
import static enumerated.Outcome.*;

enum RoShamBo6 implements Competitor<RoShamBo6> {
  PAPER, SCISSORS, ROCK;
  private static Outcome[][] table = {
    { DRAW, LOSE, WIN }, // PAPER
    { WIN, DRAW, LOSE }, // SCISSORS
    { LOSE, WIN, DRAW }, // ROCK
  };
  public Outcome compete(RoShamBo6 other) {
    return table[this.ordinal()][other.ordinal()];
  }
  public static void main(String[] args) {
    RoShamBo.play(RoShamBo6.class, 20);
  }
} ///:~
```

1056

table与前一个例子中**initRow()**方法的调用次序完全相同。

与前面一个例子相比，这个程序代码虽然简短，但表达能力却更强，部分原因是其代码更易于理解与修改，而且也更直接。不过，由于它使用的是数组，所以这种方式不太“安全”。如果使用一个大型数组，可能会不小心使用了错误的尺寸，而且，如果你的测试不能覆盖所有的可能性，有些错误可能会从你眼前溜过。

事实上，以上所有的解决方案只是各种不同类型的表罢了。不过，分析各种表的表现形式，找出最适合的那一种，还是很有价值的。注意，虽然上例是最简洁的一种解决方案，但它也是相当僵硬的方案，因为它只能针对给定的常量输入产生常量输出。然而，也没有什么特别的理由阻止你用**table**来生成功能对象。对于某类问题而言，“表驱动式编码”的概念具有非常强大的功能。

19.12 总结

虽然枚举类型本身并不是特别复杂，但我还是将本章安排在全书比较靠后的位置，这是因为，程序员可以将**enum**与Java语言的其他功能结合使用，例如多态、泛型和反射。

虽然Java中的枚举比C或C++中的**enum**更成熟，但它仍然是一个“小”功能，Java没有它也已经（虽然有点笨拙）存在很多年了。而本章正好说明了一个“小”功能所能带来的价值。有时恰恰因为它，你才能够优雅而干净地解决问题。正如我在本书中一再强调的那样，优雅与清晰很重要，正是它们区别了成功的解决方案与失败的解决方案。而失败的解决方案就是因为其他人无法理解它。

1057

关于清晰的话题，Java 1.0对术语enumeration的选择正是一个不幸的反例。对于一个专门用于从序列中选择每一个元素的对象而言，Java竟然没有使用更通用、更普遍接受的术语iterator来表示它（参见集合）。有些语言甚至将枚举的数据类型称为enumerators！Java修正了这个错误，但是**Enumeration**接口已经无法轻易地抹去了，因此它将一直存在于旧的（甚至有些新的）代码、类库以及文档中。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net购买此文档。

1058